

CLASS 10 B

One-Dimensional Arrays (Vectors)

Fundamental Concepts, Indexing, and Basic Operations

Syntax and Declaration

STRUCTURE AND SYNTAX

A one-dimensional array is a sequence of elements of the **same type**, allocated contiguously in memory:

- ▶ Allocated memory is contiguous.
- ▶ Each element is referenced by an **index**.
- ▶ Indices range from 0 to $\text{Size} - 1$.

```
// General syntax:  
baseType name[Size];  
  
// Declaration examples:  
int X[10];  
int n, A[100], m, B[50];
```

Referencing Elements

THE SUBSCRIPT OPERATOR

Accessing an element is performed via the subscript operator `[]`, which has the highest precedence:

- ▶ An indexed element is treated like a regular variable.
- ▶ **Warning:** C++ does not check if the index is within the valid range `[0, Size - 1]`.
- ▶ Going out of bounds leads to **undefined behavior**.

```
int X[10];

cin >> X[0];           // input
X[0] = 17;              //
                        assignment
cout << X[0];           // output
cout << X[0] / 5;       //
                        expressions
```

Initial Values of Elements

MEMORY MANAGEMENT

Default initialization depends directly on the declaration scope:

- ▶ **Global Array:** Elements are guaranteed to be initialized to **0**.
- ▶ **Local Array:** Elements contain **residual** / **indeterminate** values (*garbage values*).

```
int A[100]; // Global: all are
            0

int main() {
    int B[100]; // Local:
                indeterminate

    // B[0] is unpredictable!
    return 0;
}
```

Physical Size vs. Logical Size

ESSENTIAL CONCEPTS

- ▶ **Physical Size:** The maximum number of elements allocated in memory at compile time (e.g., 100).
- ▶ **Logical Size (n):** The current number of elements actively used in the algorithm.
- ▶ **Rule:** The condition $n \leq \text{Physical Size}$ must always be satisfied.

```
// Physical size: 100  
int X[100], n;  
  
// Logical size: n  
cin >> n;  
  
for(int i = 0; i < n; i++) {  
    X[i] = 1;  
}
```

Reading and Displaying an Array

BASIC OPERATIONS

One-dimensional arrays cannot be read or displayed in a single operation:

- ▶ `cin >> X;` causes a **syntax error**.
- ▶ `cout << X;` prints the **memory address** of the first element.
- ▶ Operations are performed iteratively by traversing the array.

```
cin >> n;  
// Reading  
for(int i = 0; i < n; i++)  
    cin >> X[i];  
  
// Displaying  
for(int i = 0; i < n; i++)  
    cout << X[i] << ' ';
```

0-Based Indexing vs. 1-Based Indexing

PROBLEM CONVENTIONS

Many problem statements index elements starting from 1:

- ▶ Element `V[0]` can simply be ignored.
- ▶ The active elements become `V[1]`, ..., `V[n]`.
- ▶ **Important:** The physical size must be at least 1 greater than the maximum limit for n .

```
// For n <= 100:  
int V[101], n;  
  
cin >> n;  
for(int i = 1; i <= n; i++)  
    cin >> V[i];  
  
for(int i = 1; i <= n; i++)  
    cout << V[i] << ' ';
```

Initialization at Declaration

SYNTAX

Direct initialization is permitted **only at declaration**:

- ▶ Unspecified elements are automatically filled with 0.
- ▶ `int A[10] = {1};` sets only `A[0] = 1`, with the rest being 0.
- ▶ Subsequent reassignment using brace-enclosed lists is syntactically illegal.

```
int A[] = {10, 20, 30};  
// Automatic size = 3  
  
int B[10] = {10, 20};  
// B[0]=10, B[1]=20, rest are 0  
  
int C[10] = {0};  
// All are 0  
  
// ILLEGAL later:  
// C = {0}; // Syntax error!
```